Static and Dynamic Comparison of Mutation Testing Tools for Python

Lucca Renato Guerino
Department of Computing
Federal University of São Carlos
São Carlos, São Paulo, Brazil
luccaguerino@estudante.ufscar.br

Pedro Henrique Kuroishi
Department of Computing
Federal University of São Carlos
São Carlos, São Paulo, Brazil
phk@ufscar.br

Ana Cristina Ramada Paiva
INESC TEC Faculty of Engineering University of Porto
Porto, Porto, Portugal
apaiva@fe.up.pt

Auri Marcelo Rizzo Vincenzi*
Department of Computing
Federal University of São Carlos
São Carlos, São Paulo, Brazil
auri@ufscar.br

Abstract

Context: Mutation testing is a rigorous approach for assessing the quality of test suites by injecting faults (i.e., mutants) into software under test. Tools, such as CosmicRay and Mutpy, are examples of Mutation Testing tools for Python software programs. **Problem**: With different Python mutation testing tools, comparative analysis is lacking to evaluate their effectiveness in different usage scenarios. Furthermore, the evolution of these tools makes continuous evaluation of their functionalities and characteristics necessary. **Method**: In this work, we evaluate (statically and dynamically) four Python mutation testing tools, namely CosmicRay, MutPy, MutMut, and Mutatest. In static evaluation, we introduce a comparison framework, adapted from one previously applied to Java tools, and collected information from tool documentation and developer surveys. For dynamic evaluation, we use tests built based on those produced by Pynguin, which are improved through the application of Large Language Models (LLMs) and manual analyses. Then, the adequate test suites were cross-tested among different tools to evaluate their effectiveness in killing mutants each other. **Results**: Our findings reveal that CosmicRay offers superior functionalities and customization options for mutant generation compared to its counterparts. Although CosmicRay's performance was slightly lower than MutPy in the dynamic tests, its recent updates and active community support highlight its potential for future enhancements. Cross-examination of the test suites further shows that mutation scores varied narrowly among tools, with a slight emphasis on MutPy as the most effective mutant fault model.

CCS Concepts

• **Software and its engineering → Software verification and validation**; **Empirical software validation**.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SBQS 2024, November 05–08, 2024, Salvador, Brazil
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1777-2/24/11
https://doi.org/10.1145/3701625.3701659

Keywords

Software Testing, Experimental Software Engineering, Automated Test Generation, Coverage Testing, Mutation Testing, Testing Tools, Python Mutation Tools

ACM Reference Format:
Lucca Renato Guerino, Pedro Henrique Kuroishi, Ana Cristina Ramada Paiva, and Auri Marcelo Rizzo Vincenzi. 2024. Static and Dynamic Comparison of Mutation Testing Tools for Python. In *XXIII Brazilian Symposium on Software Quality (SBQS 2024), November 05–08, 2024, Salvador, Brazil.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3701625.3701659

1 Introduction

Python has emerged as a prominent programming language over the past years. Its ranking in the Tiobe Index [18] is currently recognized as the most widely used language for applications worldwide. Ensuring the quality of Python applications is paramount, and adequate testing is relevant. One way to assess the quality of tests is through mutation testing, a rigorous approach to evaluate a set of tests by injecting known faults into the code.

There are several mutation testing tools for Python programs. However, determining the best tool depends on several factors. This paper aims to evaluate and compare four tools from two distinct perspectives. The first perspective utilizes a static framework previously employed in the assessment of Java mutation testing tools [1], adapted for this study. Building on the work by Amalfitano et al. [1], we will also undertake a dynamic evaluation of these tools. This involves building test suites suitable for each tool, that is, capable of killing mutants generated by that tool, and then using that adequate test suite to evaluate whether they are also capable of killing mutants generated by the other tools. This paper aims to compare four mutation testing tools for Python: MutPy [6], MutMut [7], Mutatest [8], and Cosmic-Ray [3].

Inspired by the work of Guerino and Vincenzi [5], we decided to use their publicly available benchmark. Complementing Pynguin's generated test sets with manual and LLM support, we generate an adequate test set for each mutation testing tool and each subject program, ensuring an even higher degree of software reliability and robustness. We also perform a static assessment of the current state of mutation testing tools and their capabilities by using an adapted version of the framework proposed by [1].

Building upon the exploration of Pynguin's algorithms for unit test generation, our study enhances "all-smart" test sets by integrating advanced Large Language Models (LLMs) such as ChatGPT[1], Gemini[2], and Bing[3]. We aim to evaluate the use of the LLM to improve and expand test suites, seeking higher test coverage and mutation scores as part of the dynamic analysis proposed in this paper. If necessary, we complement the test suite by manually designing test cases to achieve adequate test sets for each mutation testing tool under consideration.

Thus, the contributions of this paper can be summarized as follows:

- To gather information surrounding the mutation tools to establish which is the best at testing the selected Python programs.
- To instantiate a previously developed framework [1] in another context for static evaluation of the selected mutation testing tools for Python;
- To generate adequate test sets by using test generator, LLM, and when necessary, manually, for a set of 21 programs and 4 different mutation testing tools for Python, summing up 84 adequate test sets, making them publicly available for further experiments[4];
- To perform a dynamic evaluation of the mutation testing tools by cross-checking the capability of an adequate test set for a given tool on killing mutants of the others.

The remainder of this document is outlined as follows: Section 2 provides the necessary background to comprehend this study. Section 3 discusses works related to ours, highlighting the differences. Section 4 outlines the experiment's design, including the test sets' current standing and methods of improving them. Section 5 details the data gathered and its subsequent analysis. The implications of the analyzed data are deliberated in Section 6. Potential limitations and their possible effects on the experiment's outcomes are addressed in Section 7. The paper is wrapped up in Section 8, where we suggest avenues for future research based on the findings and data obtained from this study.

2 Background

This section will explain relevant software testing and related concepts so that the rest of the paper can be understood.

2.1 Mutation Testing

Software testing is a critical process in the development life-cycle that involves evaluating a software application to ensure it meets the specified requirements and functions correctly. Mutation testing is an important technique in software testing designed to evaluate the effectiveness of test suites. It involves introducing small, systematic modifications to a program's source code to create a series of mutant versions. This approach helps determine whether the existing test cases can detect these modifications, thereby assessing the test suite's ability to detect faults.

The effectiveness of the test suite is quantified using the mutation score, which is the ratio of the number of mutants detected (killed mutants) to the total number of non-equivalent mutants generated. Mutants that are detected and cause the test suite to fail are termed "killed mutants," while those that pass the test suite undetected are referred to as "live mutants." A higher mutation score indicates a more effective test suite, suggesting an excellent capability to identify and address potential faults in the software [2, 15].

The automation of mutation testing has seen significant advancements with the development of various tools designed to optimize this task (MutPy[5], MutMut[6], Mutatest[7], Cosmic-Ray[8] and Pytest Mutagen[9]). These tools automate the process of generating mutants and running test suites against them, speeding up the mutation testing process and enhancing its accuracy and repeatability. By employing these automated systems, practitioners can efficiently assess the effectiveness of their test suites in detecting faults, thereby improving the robustness of the software, a very important demand of modern software development cycles.

It is essential to consider aspects that differentiate these tools, such as their mutation operators, the number of mutants generated, and their resulting data processing methods. Mutation operators create mutants by making specific code changes, and the diversity of these operators can influence the scope and depth of the testing. The number of mutants produced is also crucial; too many can cause excessive processing times, while too few may not test the software's robustness adequately. Lastly, the analysis and interpretation of test results vary among tools, affecting the insights and utility of the testing process.

2.2 Automated Test Data Generation

A fundamental component of testing is the generation of test cases. Automatic test case generation is a pivotal technique in software testing, leveraging algorithms and tools to produce test scenarios without manual intervention. This approach streamlines the testing process and enhances comprehensiveness by systematically covering various input scenarios and program paths. By automating test generation, developers can efficiently identify potential defects and ensure higher software quality, significantly reducing the time and effort required compared to manual test creation.

As such, automated test generation continues to pose challenges [20], particularly for Python, because of its dynamic typing system. In Python, variables do not require declaration before use, and their types can change during the execution of the code. This dynamic nature complicates identifying patterns necessary for generating automated tests for specific programs [4, 11].

In the context of enhancing software testing methodologies, Search-based Software Testing (SBST) is a prominent strategy that uses meta-heuristic search techniques to optimize test case generation [13]. By framing test generation as an optimization problem, SBST aims to maximize code coverage or fault detection using genetic algorithms, simulated annealing, or hill-climbing algorithms. This approach efficiently navigates potential test inputs to enhance

[1] https://chat.openai.com/
[2] https://gemini.google.com/
[3] https://www.bing.com/
[4] https://github.com/aurimrv/python_experiments2
[5] https://github.com/mutpy/mutpy
[6] https://github.com/boxed/mutmut
[7] https://github.com/EvanKepner/mutatest
[8] https://github.com/sixty-north/cosmic-ray
[9] https://github.com/timpaquatte/pytest-mutagen

software testing efficacy, especially in complex systems where manual testing is impractical, thus improving the efficiency and effectiveness of testing processes.

Several tools have been developed to facilitate the automatic generation of test cases in Python, each adopting a distinct approach to tackle this challenge. Tools such as Faker[10], Pynguin[11], Ponicode[12], Hypothesis[13], Auger[14], and TestData[15] exemplify the variety of strategies employed. These tools differ in their underlying methodologies, from employing sophisticated algorithms that mimic user behavior to using frameworks that generate data-driven tests. This diversity in approaches not only underscores the dynamic nature of software testing but also enhances the adaptability of test suites to various application domains, thereby improving the efficacy and efficiency of the testing process.

In the realm of automated test generation for Python, Pynguin represents a significant advancement, as discussed in a detailed study by Lukasczyk et al. [12]. This tool leverages mutation-based generation methods to construct comprehensive unit tests effectively. Pynguin is particularly noted for its capability to interpret Python bytecode and apply evolutionary algorithms, which aids in producing robust and effective test suites. This enables developers to improve code coverage significantly, enhancing software applications' quality and reliability. The article elaborates on Pynguin's application to real-world Python projects, demonstrating its practical effectiveness and potential to streamline the testing process within development cycles.

Large Language Models (LLMs) are changing software testing by augmenting existing test suites and generating novel test cases [10, 16, 22, 23]. These models leverage their deep understanding of code semantics to enhance test coverage by identifying overlooked scenarios and developing tests for edge cases. LLMs can also refine and optimize test scripts, improving their effectiveness in detecting subtle bugs. This dual capability of LLMs to innovate and improve test procedures significantly enhances the efficiency and robustness of software testing.

While the LLM has successfully generated test cases, it still has some limitations depending on the program structure and complexity. Therefore, it is essential to carefully evaluate the results generated by the LLM and manually verify that all scenarios have been covered. Nonetheless, the LLM is a helpful tool for generating test cases and has the potential to improve the efficiency and effectiveness of software testing significantly.

3 Related Work

In their comprehensive study, Amalfitano et al. [1] distinguish themselves by offering an exhaustive comparison of Java mutation testing tools. Their framework for evaluating these tools spans five critical dimensions—Version, Deployment, Mutation Process, User-centric Features, and Mutation Operators—providing an unparalleled depth of analysis. Furthermore, their engagement with stakeholders through targeted surveys sheds light on the factors influencing tool selection among researchers, educators, and practitioners. This approach underscores the multifaceted nature of mutation testing tools' evaluation. It marks a significant leap forward in guiding users to make informed decisions based on specific needs, thus filling a notable gap in the software testing literature.

Márki and Lindström [14] empirically compared Java mutation tools – Major, MuJava, and PIT—examining mutators' efficiency, effectiveness, and user adaptability. Their study employs cross-testing to reveal the unique strengths of each tool, particularly in fault detection and user experience, providing valuable guidance for selecting and integrating mutation testing tools in software development environments. Meanwhile, Kintis et al. [9] performed a comparative analysis of Java mutation testing tools, highlighting PITRV as the most effective at killing mutants with broad coverage, while MuJava excelled in more focused, context-specific applications. Major and PIT also performed competitively, but no single tool dominated across all metrics, suggesting that leveraging multiple tools can ensure more comprehensive test coverage and improve software robustness.

Uzunbayir and Kurtel [19] evaluate mutation testing tools for C#, focusing on Nester, Stryker, NinjaTurtles, VisualMutator, PexMutator, and CREAM. The study analyzes their support for traditional and object-oriented mutation operators, unit testing integration, and usability features like GUIs and documentation. It also examines their efficiency in generating and managing mutants, including handling equivalent mutants. This analysis fills a gap in C# mutation testing literature, guiding developers in selecting tools to enhance testing and integration in C# environments.

The study from Tian et al. [17] investigates using LLMs to enhance the detection of equivalent mutants in software testing. It focuses on how LLMs can improve the efficiency of identifying these mutants, which pose challenges in mutation testing due to their non-observable impact on program behavior. The paper outlines limitations in LLM-driven approaches and offers insights for future advancements in automating equivalent mutant detection using machine learning techniques.

Wang et al. [21] provide a survey of LLMs in software testing, highlighting key areas from 102 studies from unit til system test generation, program repair, different LLM models, types of prompt engineering, and incorporating other tech with LLM. Our work builds on this foundation by focusing on unit test generation, particularly exploring ChatGPT's capabilities within zero-shot and few-shot learning frameworks for unit test case generation, which are present in many of the analyzed works from the survey.

4 Experiment Design

The experiment aims to determine which Python mutation testing tool can generate the most relevant mutants for fault detection. The tools comparison includes a static analysis without executing them and a dynamic analysis, where an assessment is made through their execution. Python is a popular programming language with different testing tools to support testing automation. Concerning mutation testing tools, we searched on PyPi[16] and GitHub[17] repositories and selected the four most used and updated

[10] https://github.com/joke2k/faker
[11] https://github.com/se2p/pynguin
[12] https://github.com/ponicode
[13] https://github.com/HypothesisWorks/hypothesis
[14] https://github.com/apennisi/auger-python
[15] https://github.com/Jaymon/testdata
[16] https://pypi.org/
[17] https://github.com/topics/mutation-testing?l=python

tools, considering GitHub activity (last resolved issue in the last four years): MutPy [6], MutMut [7], Mutatest [8], and Cosmic-Ray [3].

In the static evaluation of these mutation testing tools, detailed assessments focus on their theoretical capabilities and implementations through the review of the available documentation. The evaluation encompasses a comparative analysis of how each tool designs and generates mutants based on the syntax and semantics of the Python language without actually running the tool.

Additionally, data regarding the development and updates of the tools are scrutinized to determine their current relevance and support. For this study, the static properties of the four main tools under consideration are compared, highlighting differences in their approach to mutant generation and their adaptability to different programming practices.

Building upon the framework established by Amalfitano et al. [1], which effectively compares Java mutation tools across multiple dimensions, our study proposes an adapted version of this framework to assess mutation testing tools for Python. This adaptation acknowledges the unique aspects of Python's dynamic typing and runtime behavior compared to Java's static type system. This framework also considers different perspectives of interest in mutation tool usage, such as education, research, and practice.

By adapting the existing Java framework [1] to a Python-specific one, we aim to provide an analysis that evaluates each tool's ability to generate meaningful mutants and process mutation data effectively. Additionally, a questionnaire was sent to developers to supplement missing details in the documentation. This survey aims to collect essential information directly from the tool creators, ensuring our analysis includes accurate data on each tool's functionality and usability.

For the dynamic evaluation of mutation testing tools, we use a set of Python programs, selected by convenience, once this set of programs is publicly available in a GitHub repository[18], referenced in [5]. Moreover, we use the best test set available in the repository for each program (*all-smart* test set) as the starting point for the dynamic comparison of the selected mutation testing tools. Since such test sets are not 100% adequate, we evolved them using LLM and manual tests to construct adequate test suites capable of killing all non-equivalent mutants produced by the four tools under analysis.

The Python programs used in this experiment are listed in Table 1. With these programs, we associate an identification (ID). The column Program displays the name of each program, and columns CL, ME, CC, and SLOC present the number of classes, methods, cyclomatic complexity, and source lines of code for each program.

One might argue that the chosen programs do not mirror those typically encountered in real-world applications. Nevertheless, it is crucial to recognize that, as was the objective in the source of this selection [5], they assess the efficacy of test generator algorithms that operate at the method or function level. These programs correspond to different implementations of traditional data structures or sorting algorithms, making them adequate for educational purposes in an experimental package about software testing and mutation testing. Moreover, our premise is that a mutation tool with lower performance on small programs will not have a better performance on more extensive programs.

[18]https://github.com/aurimrv/python_experiments

Table 1: Set of utilized programs adapted from [5]

ID	PROGRAM	CL	ME	CC	SLOC
P01	binarySearchTree1.py	2	16	4	109
P02	binarySearchTree2.py	2	15	3	144
P03	binarySearchTree3.py	5	35	5	248
P04	binarySearchTree4.py	2	5	4	36
P05	identifier1.py	1	3	8	31
P06	linkedList1.py	4	14	4	118
P07	linkedList2.py	2	9	4	53
P08	linkedList3.py	2	10	2	61
P09	linkedList4.py	2	8	3	50
P10	linkedList5.py	2	11	4	85
P11	queue1.py	2	8	2	39
P12	queue2.py	3	16	4	92
P13	queue3.py	3	20	2	117
P14	queue4.py	3	14	5	97
P15	queue5.py	2	4	4	26
P16	sort1.py	1	6	4	59
P17	stack1.py	2	6	2	76
P18	stack2.py	1	5	2	13
P19	stack3.py	3	17	2	76
P20	stack4.py	3	11	3	57
P21	stack5.py	2	6	2	19
AVG		2.3	11.4	3.5	76.5
SD		1.0	7.3	1.5	53.1

To derive these adequate test sets, we augmented the existing test suites with LLMs, namely ChatGPT 3.5, to fill in any gaps where the automated tests lacked 100% of coverage and mutation score. This usage consisted of providing the code that should be tested for the initial creation of tests covering all classes. From there, mutation testing was applied, and the surviving mutants were fed to the LLM interactively to create more tests to kill the live mutant. The prompts used are present in the `README.md` file inside the project repository[19]. Essentially, two types of prompts were created: a generic, which provided the entire application source code and requested tests to cover all code, and a specific one that requested tests to kill each live mutant after analyzing the first test set's coverage and mutation score.

When the LLM's capabilities were insufficient, manual test creation was used to ensure the completeness and accuracy of the test suites. Additionally, equivalent mutants, which do not alter the program's external behavior and are undetectable by the tests, were identified and segregated manually. This blended approach ensures a thorough and precise evaluation process, enhancing the reliability of our comparative analysis of mutation testing tools.

By leveraging adequate test suites, we aim to measure and compare the effectiveness of each tool in a controlled environment. This methodology allows for a detailed assessment of each mutation testing tool's performance, detecting and addressing all possible mutations each tool generates across different source codes.

Figure 1 partially depicts the dynamic analysis, complementing static analysis. The gray boxes highlight the effort of analyzing each mutation testing tool's live mutants, resulting from executing the Pynguin test set generated from all algorithms implemented in Pynguin, except the Random Algorithm. We identify this test set by TS_{AS}. After the analysis of live mutants, with the support of the LLMs to generate additional test cases, we complement the TS_{AS} for each specific tool, such that we create an adequate test set for each tool and each subject program in our study, also identifying

[19]https://github.com/aurimrv/python_experiments2

the equivalent mutants. For each subject program, the individual adequate generated test set is determined by TS_{MP}, TS_{MT}, TS_{MM}, and TS_{CR}, and the set of equivalent mutants as EM_{MP}, EM_{MT}, EM_{MM}, and EM_{CR}.

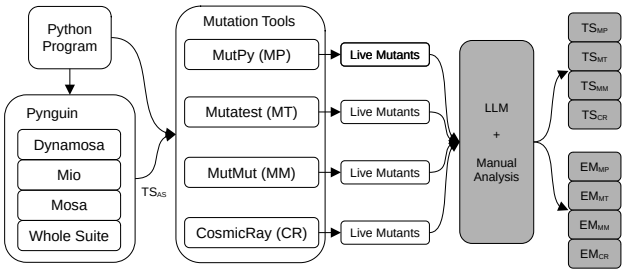


Figure 1: Experiment Design Diagram

5 Data Collection and Analysis

In this section, we detail the process of data collection and analysis. In Section 5.1, we describe the process for static analysis, and in Section 5.2 for the dynamic analysis.

5.1 Static Collected Data

In the static analysis of the testing tools, data collection was conducted by extracting information from both the official documentation of the tools (as well as related research detailing the tools) and a structured Google Forms questionnaire[20] submitted to tools' developers. The idea is that the questionnaire conforms or complements information that may be missing in the documentation of the tools. The checklist and questionnaire used for this task were adapted from the work of [1]. The adaptation was carried out by the first author and validated by the other authors of the paper.

The checklist was first filled out by the paper's first author and then validated by the last authors once both had experience using all the analyzed tools.

The questionnaire was submitted approximately two months before the experiment's conclusion for at least two developers of each tool obtained from the official GitHub repository of each tool; however, only two responses were received from developers of MutMut and CosmicRay, which is understandable considering these are the most recently updated tools. It served the purpose of providing additional information that may be missing from the tools' documentation and also allowing a greater understanding of their internal workings. Again, the first and last authors of the paper consolidated the answers to the questionnaire.

This dual approach ensured a comprehensive data aggregation, capturing detailed insights about each tool's functionalities, limitations, and operational contexts.

The collected information from the checklist and questionnaire is organized into Table 2, facilitating a clear and accessible presentation of the attributes and capabilities of each testing tool. This tabular compilation is crucial for evaluating the tools' theoretical and practical applications, providing a robust foundation for subsequent comparative analysis.

License: This indicates the type of open-source license under which each tool is distributed. For example, MutPy is distributed under the Apache 2 license, allowing considerable usage and redistribution freedom. Both Mutatest and CosmicRay use the MIT license, which is similarly permissive.

Release version: The specific version of each currently available tool indicates the tools' maturity and development stage.

Release year: The year when the current release of each tool was made available. This provides insight into the tool's development and evolution to more recent technologies. MutMut and CosmicRay are highlighted as the most recently updated.

Last resolved issue: Indicates the most recent year in which an issue was officially resolved, reflecting the active maintenance and support of the tool. MutMut is the most active at the time of this analysis.

Python version: Lists the versions of Python compatible with each tool, which is crucial for developers to ensure compatibility with their projects.

Build-tool integration: This attribute specifies whether the mutation tool can be integrated with build tools, which is essential for automating the mutation testing process in a development pipeline. Cosmic Ray is the only one identified through the survey with this capability.

Testing framework: Identifies the testing frameworks supported by each tool, such as Unittest and Pytest, which are popular in the Python community.

Mutation level: Categorizes the tool's operating level, such as modifying byte code or source code, impacting how deeply and in what ways the tool can introduce changes. All of the analyzed tools only mutate on the source code level.

Test selection: Differentiates between automated and manual test selection mechanisms provided by each tool, affecting how test cases are chosen for mutation testing, but all are manual.

Mutation operator selection: Refers to the ability to choose specific mutation operators, allowing users to customize which kinds of mutations are applied during testing. Out of all of them, CosmicRay is highlighted as the one that can also select operator classes instead of only the individual operators.

Mutant inspection: Details the features provided to inspect mutation impacts, such as viewing changes over lines of code (LOC) and assessing live mutants. MutMut is the only one that hides killed mutants, showing only live mutants over LOC.

Kill matrix: A feature only CosmicRay offers that maps tests to mutants they kill, functional for detailed analysis of test effectiveness.

Analysis runtime reduction: Indicates if the tool provides features to reduce the time for mutation analysis.

User interface: Specifies the kinds of interfaces available (CLI, GUI, IDE), impacting how users interact with the tool, although everyone only includes CLI.

Required inputs: Lists the types of inputs each tool requires, such as source code or test cases, necessary for performing mutation testing. None of the tools stand out in this category.

[20] https://forms.gle/X1MpNLn6Rr1PK9S86

Table 2: Static Framework

	Mutation Tool			
Attribute	**MutPy**	**Mutatest**	**MutMut**	**CosmicRay**
Version				
Licence	Apache 2	MIT	BSD-3	MIT
Release version	"0.6.1"	"3.1.0"	"2.4.4"	"8.3.7"
Release year	2019	2022	2023	2023
Last resolved issue	2021	2023	2024	2023
Deployment				
Python version	Python 3.3+	Python 3.7+	Python 3.7+	Python 3.5+
Build-tool integration	-	-	-	✓
Testing framework	Unittest & Pytest	Unittest & Pytest	Unittest & Pytest	Unittest & Pytest
Mutation process				
Mutation level				
Byte code	-	-	-	-
Source code	✓	✓	✓	✓
Test selection				
Automated	-	-	-	-
Manual	✓	✓	✓	✓
Manual (GUI)	-	-	-	-
Mutation operator selection				
Operators	✓	✓	✓	✓
Operator classes	-	-	-	✓
Mutant inspection				
Over LOC	✓	✓	-	✓
Over Live Mutants LOC	-	-	✓	-
Side by side	-	-	-	-
Kill matrix				
Test method	-	-	-	-
Test class	-	-	-	✓
Analysis runtime reduction	-	-	-	-
User-centric features				
User interface				
CLI	✓	✓	✓	✓
GUI	-	-	-	-
IDE	-	-	-	-
Required inputs				
Byte code	-	-	-	-
Source code	✓	✓	✓	✓
Test cases	✓	✓	✓	✓
Produced outputs				
Mutated source code	-	-	-	✓
Mutated byte code	-	-	-	-
Summary report	✓	✓	✓	✓
Reduced test suite	-	-	-	-
Documentation quality	Insufficient	Good	Sufficient	Sufficient
Mutation operators				
Data types				
Primitives	✓	✓	✓	✓
Arrays & classes	✓	-	-	✓
Pre-defined operators				
Assignment	✓	✓	✓	✓
Arithmetic	✓	✓	✓	✓
Unary	✓	-	✓	✓
Relational	✓	✓	✓	✓
Conditional	✓	✓	✓	✓
Bit-level	✓	✓	✓	-
Python specific				
Decorators	✓	-	-	✓
Pass	✓	-	-	-
Final Score	18	14	15	20

Produced outputs: Details each tool's outputs, such as mutated source code or summary reports, which are essential for evaluating mutation testing results, where CosmicRay shows itself a little better than the others.

Documentation quality: Describes the comprehensiveness and clarity of the tool's documentation, which is essential for user understanding and practical usage. In this case, MutPy stands out as the worst of them, with not much content available online to be analyzed, because, unlike other tools, it does not have a 'Read the Docs' page or similar containing comprehensive documentation, but only a brief operational description on its GitHub page.

Data types: Indicates the kinds of data types the tools are capable of mutating, such as primitives or arrays and classes, affecting the scope of the testing.

Pre-defined operators: Considers each tool's already implemented mutant operators. Most tools are equivalent here, with little to differentiate between them.

Python specific: Notes any mutation operators or features specifically designed for Python, such as decorators or the pass statement. Only MutPy and CosmicRay show any capability of these mutant implementations.

Section 6.1 discusses the static collected data.

5.2 Dynamic Collected Data

In the dynamic analysis of the selected testing tools, data collection involved generating adequate test suites, initially generated by Pynguin, which combined all the algorithms (excluding Random) from the tool. Detailed information about how these Pynguin generated test sets can be found elsewhere [5].

Pynguin's ability to create diverse test cases using a variety of algorithms allows for a robust examination of software behavior under different testing scenarios. Nevertheless, from [5] work, we know the best Pynguin-generated tests are not able to kill all non-equivalent mutants of all mutation testing tools. Consequently, this dynamic evaluation will provide deeper insights into the strengths and weaknesses of the automated testing processes implemented by Pynguin. The idea is to enhance Pynguin generate test sets until they become adequate for each individual mutation testing tool.

This enhancement was accomplished by combining LLMs and manual analysis to refine and expand the test cases. This approach aimed to cover faults and scenarios that Pynguin could not cover. Information regarding the number of tests generated for each tool can be found in Table 3, which includes the tests automatically generated by Pynguin in the second column and, in the subsequent columns, the appropriate number of additional tests we create to have an adequate test set for each subject program and each mutation testing tool.

Table 3: Adequate Test Suite Size

		LLM + Manual Tests			
ID	Pynguin	MutPy	Mutatest	MutMut	Cosmic-Ray
P01	117	10	5	3	4
P02	67	12	7	5	5
P03	157	23	18	14	16
P04	38	3	6	1	0
P05	46	6	3	3	2
P06	111	6	1	1	3
P07	63	4	5	4	2
P08	44	2	2	3	3
P09	56	3	3	4	5
P10	125	6	3	3	3
P11	33	7	4	4	0
P12	105	8	7	6	2
P13	70	4	3	2	1
P14	80	13	7	8	3
P15	24	2	1	1	0
P16	54	5	5	4	5
P17	59	15	12	7	8
P18	21	2	1	1	0
P19	64	7	3	4	3
P20	62	9	7	5	3
P21	25	3	4	3	1
AVG	67.67	7.14	5.1	4.1	3.29
SD	36.48	5.21	3.96	2.96	3.55

For instance, considering program P01, MutPy-adequate test set comprises 117 test cases from Pynguin plus 10 additional tests created to kill MutPy specific live mutants. In the same way, Cosmic-Ray-adequate test set comprises 117 test cases from Pynguin plus 4 additional tests created to kill Cosmic-Ray specific live mutants.

Moreover, an adequate test set is necessary for each of the 21 subject programs to analyze the live mutants and identify the equivalent ones. We carried out this analysis and determined all equivalent mutants manually. This manual analysis was made through a paired analysis of the authors. At first, a mutant was subject to be considered equivalent, and then it was sent to review by the second author. The complete datasets, including the total number of mutants generated by each tool for each subject and the corresponding number of equivalent mutants identified, have been compiled into Table 4. This table overviews the mutation testing results for cross-examining the tools. In the subsequent phase of our analysis, after determining adequate test suites for each mutation testing tool, a cross-examination was conducted.

Each adequate test suite of each tool was evaluated against mutants generated by its own and cross-executed against mutants produced by the other tools. This method allowed us to assess the robustness and strength of each test suite across different mutation environments. In this manner, in Table 5, the rows represent the tools being executed on the appropriate tests, and the columns depict the performance of these tests on each tool.

In Table 5, MP, MT, MM, and CR are short for MutPy, Mutatest, MutMut, and Cosmic-Ray. Scores highlighted in gray represent the mutation scores obtained when a test suite was run against mutants from its generating tool; these scores were calculated manually, eliminating the mutants considered equivalent, as no tools allow setting a mutant as equivalent and excluding them from the mutation score automatically.

Conversely, scores highlighted in yellow denote the highest mutation scores achieved when test suites were executed on a different tool from which they were generated. This cross-tool effectiveness underscores the adaptability and thoroughness of the test suites in identifying faults across varied mutation frameworks.

Furthermore, each row and column's means and standard deviations were calculated. For the rows, it is possible to determine which tool generates the most difficult mutants to kill. At the same time, for the columns, one can identify the appropriate test set that performs best across other tools.

6 Discussion

6.1 Static Evaluation

The criterion used for the static point of view was a scoring system, where each element present in the tool is worth one point, and each tool has its own score. The final score is presented in the last line of Table 2. Elements such as the quality of documentation and recent updates would serve as tiebreakers.

Our evaluation revealed that, from the static evaluation point of view, Cosmic-Ray distinguishes itself with superior functionalities and extensive customization options for generating mutants. The tool offers a robust framework that enables detailed configuration of test parameters, facilitating a more tailored and effective testing process.

Table 4: Equivalent Mutant Data

ID	MutPy		MutMut		Mutatest		CosmicRay	
ID	Total	Equivalent	Total	Equivalent	Total	Equivalent	Total	Equivalent
P01	239	8	72	3	200	11	211	13
P02	222	11	86	10	216	21	188	10
P03	511	5	179	4	289	19	291	22
P04	53	0	17	1	40	0	18	0
P05	100	3	61	5	114	6	140	8
P06	199	11	95	1	122	5	120	3
P07	107	1	45	0	96	5	44	1
P08	116	4	48	5	48	5	46	4
P09	110	0	46	0	62	1	72	0
P10	136	5	54	1	111	10	74	2
P11	67	0	27	0	33	0	14	0
P12	186	2	73	0	149	11	62	2
P13	218	11	93	10	116	18	83	10
P14	198	3	74	0	117	5	112	7
P15	48	2	20	2	37	2	16	3
P16	115	4	61	3	116	9	185	14
P17	129	0	68	0	99	5	114	3
P18	29	0	6	0	13	1	19	1
P19	150	2	55	5	59	8	62	5
P20	122	0	47	0	60	2	72	3
P21	35	0	9	0	19	0	5	0
AVG	147.14	3.43	58.86	2.38	100.76	6.86	92.76	5.29
SD	101.93	3.74	36.87	3.06	67.89	6.16	73.43	5.62
TOTAL	3090	72	1236	50	2116	144	1948	111

Table 5: Cross-Examination Score

Tool	TS_{MP}	TS_{MT}	TS_{MM}	TS_{CR}	AVG	SD
MP	100	97.04	96.97	96.65	97.67	1.57
MT	97.92	100	97.26	96.46	97.91	1.52
MM	98.38	96.44	100	95.23	97.51	2.11
CR	99.38	99.08	98.72	100	99.3	0.54
AVG	98.92	98.14	98.24	97.09	-	-
SD	0.94	1.68	1.4	2.04	-	-

It is also noteworthy that, compared to MutPy and Mutatest, both Cosmic-Ray and MutMut have been updated more frequently. These updates are significant as they suggest active communities that continuously contribute to enhancing the tools. Such development efforts not only introduce new functionalities but also address existing bugs, thus improving the reliability and efficacy of the tools, which is essential for the sustainability and advancement of mutation testing tools.

Additionally, the static aspects that highlight CosmicRay's strengths open up opportunities for improvement and enhanced customization. This potential extends to both the generation of mutants and the use of mutation reports for diverse analyses.

These capabilities suggest that while current performance metrics may have limitations (see Section 6.2), CosmicRay's inherent flexibility and advanced features provide a solid foundation for refining the tool's effectiveness in mutation testing.

Due to these highlighted qualities and the fact that it has the most active community, Cosmic-Ray can be considered the most suitable mutation tool from a static perspective. Based on its reports, it offers greater customization in generating mutants and enhanced analysis capabilities. It is also not too far behind in the dynamic evaluation, as seen in the next section.

However, it is also essential to consider the static aspects lacking in all tools compared to the framework applied to Java tools [1]. Notably, more comprehensive user interfaces are lacking since all are controlled directly via the terminal. There is also an absence of analysis runtime reduction strategies and the capability to automatically select tests through the tools.

Another deficiency, present only in Cosmic-Ray in a primitive form, is a Kill Matrix that quickly shows which tests have killed which mutant. Moreover, none of the tools can mark equivalent mutants, which could improve the precision of the mutation score, nor do they provide reduced test sets, excluding tests that might be considered redundant. These aspects are essential when considering the future of automatically generating Python tests or evaluating different mutation testing strategies to optimize mutation testing in production software development environments.

Considering the static aspects, two tools stand out as the ones with the poorest performances: Mutatest and MutMut. Despite being updated more recently than MutPy, for example, judging by what was compared in Table 2, their customization features and test generation capabilities are met equally or better by the other tools.

6.2 Dynamic Evaluation

By creating adequate test sets, which enabled data extraction on equivalent mutants from Table 4, we can consider possible relationships between the number of mutants generated and how many are equivalent, depending on the tool.

MutPy generated the highest number of mutants. Yet Mutatest has the highest relative number of equivalent mutants, while MutPy has the lowest relative quantity of equivalent mutants.

From this, we can infer that statistically, equivalent mutants are more likely to be obtained when generated with Mutatest, whereas the opposite can be observed with MutPy.

Regarding the dynamic analysis, it emerged from Table 5 that among the four mutation testing tools evaluated, MutPy distinctly

outperformed the others. However, it is essential to note the relatively narrow range of the differences in mutation scores among the tools.

The percentage variation in mutation scores between the best and worst-performing test suites in the cross-examination phase spanned from as little as 0.39% to a maximum of 3.15%. This marginal disparity underscores that MutPy exhibited superior performance, which is also highlighted by the average of the rows, where the overall efficacy of the tools is closely matched (see Table 5).

This observation suggests that all four tools possess a comparable potential in effectively detecting mutants under varied testing scenarios. However, MutPy shows a slight edge in effectiveness compared to the current dataset and testing methodologies employed in this study. This close range of outcomes highlights the competitive nature of these tools. It suggests that selection among them could be influenced more by specific user requirements and context-specific factors rather than a broad superiority of one tool over the others. It is also possible to infer from the averages of the lines that the mutants from each tool are very similar in difficulty, with MutPy, MutMut, and Mutatest showing cross scores varying by only 0.4%. Only Cosmic-Ray stands out as generating mutants that are easier to kill, with an average of 99.3% across all tools.

In Table 6, we present the performance of the adequate test suite from all the algorithms when applied to other mutation testing tools. As highlighted in Table 5, Cosmic-Ray has the worst performing adequate test, with almost no tests that achieve a 100% score. We can also examine the data of live mutants with the appropriate test set from Mutatest and from MutMut to observe their performances, which, although slightly better than Cosmic-Ray, do not stand out among the tools. The final consideration is in relation to when the adequate test suite for MutPy was executed against the mutants generated by the other mutation testing tools. So, it is possible to extract how close this test suite was to match the other tool's ideal test suite, the one with the most zeroes in its columns.

Given this dynamic approach, it is essential to note that in practice, equivalent mutants are typically discarded as they are not crucial for detecting real application faults. However, considering this, mutation tools must be able to identify these mutants and display mutation scores with and without equivalent mutants. Moreover, identifying equivalent mutants is of fundamental importance for creating historical data about equivalence to further subsidies for developing automatic strategies for detecting equivalent mutants.

In light of this, although briefly mentioned before, the tool that stood out negatively in the dynamic analysis was Cosmic-Ray, which has already been examined more thoroughly considering static aspects. Therefore, it is conceivable that better performance could be achieved with enhanced customization capabilities for mutation operators and generated reports. However, based on the study conducted with the default settings, its performance in killing mutants from the other tools was the lowest.

It is also evident from Table 3 that the results of the best and worst mutation score performance correspond to what is expected from the size of test suites considered adequate for each tool, indicating that more tests are required for the mutants from MutPy than for those from Cosmic-Ray, which is reflected in the performance previously described, while MutMut and Mutatest stand in the middle both in size and score.

6.3 Perspective Evaluation

It is essential to consider the importance of the analyzed criteria from different perspectives. We consider the same perspectives identified by Amalfitano et al. [1], i.e., education, research, and practice perspectives. Observe that each perspective may demand different characteristics in mutation testing tools; some features may conflict, and a trade-off is necessary to determine the best tool to meet more than one perspective.

From an educational perspective, the ease of installation generally stands out as a positive attribute across the mutation testing tools evaluated. However, a notable deficiency is the lack of Graphical User Interfaces (GUIs), which hampers usability and can be a barrier for learners unfamiliar with command-line interfaces. Additionally, the availability of interactive and easily understandable reports is highly valued in educational settings. Tools like MutPy and Cosmic-Ray excel in providing such features, enhancing the learning experience by making outcomes more accessible and engaging. Conversely, MutMut and Mutatest fall short in this area, lacking intuitive report features that could otherwise aid in the educational use of these tools.

For research applications, the critical attributes of a mutation testing tool include its ease of configuration and the availability of a detailed killing matrix. These features are indispensable for researchers needing to tailor testing processes and analyze specific aspects of how tests interact with mutants. Easy analysis of live mutants is also crucial as it aids in understanding the resilience of code against testing and identifying potential areas for improvement in test coverage. Cosmic-Ray is the most suitable tool for research due to its superior configuration capability and comprehensive analysis features, facilitating a deeper investigation into mutant behavior and test effectiveness.

In practice, within the industry, the focus shifts towards the efficacy of tools in killing mutants and the speed at which tests can be executed. Here, the main concern is identifying live mutants and ensuring that the mutation testing process is efficient and integrates seamlessly into continuous integration workflows. MutPy operators represent a good fault model, and they run fast, which is crucial for quick and practical testing. However, a common limitation is the lack of runtime reduction mechanisms, like control-flow analysis, to skip irrelevant test cases. These features are essential for speeding up mutation testing in fast-paced development, indicating a need for future improvements in these tools.

7 Threats to Validity

Several threats to the validity of our findings need to be acknowledged in determining which mutation testing tool is most suitable for Python applications.

In the static evaluation, most information unavailable in the official documentation was derived from firsthand experience with the tools. This subjective data collection might introduce biases or inaccuracies due to users' varying familiarity and expertise.

Additionally, the developer questionnaire designed to supplement missing documentation details received limited responses. This low engagement could result in a lack of comprehensive insight into each tool's capabilities and limitations, potentially skewing the assessment towards incomplete data.

Table 6: Live Mutant Data

ID	MutPy-adequate test set *versus*			MutMut-adequate test set *versus*			Mutatest-adequate test set *versus*			CosmicRay-adequate test set *versus*		
	Mutatest	MutMut	CosmicRay	MutPy	Mutatest	CosmicRay	MutPy	MutMut	CosmicRay	MutPy	MutMut	Mutatest
P01	0	1	0	26	5	5	8	3	0	9	3	0
P02	3	0	0	5	5	3	6	0	0	6	1	3
P03	5	0	0	23	8	4	14	4	2	10	5	6
P04	2	1	0	2	2	0	4	0	0	4	1	2
P05	0	0	0	3	1	0	3	1	0	4	1	1
P06	0	1	2	6	0	3	6	2	3	5	2	2
P07	0	0	0	0	0	0	0	0	0	3	3	4
P08	3	1	1	0	1	0	6	3	1	1	1	0
P09	2	2	3	1	2	3	2	3	3	2	3	2
P10	6	1	3	4	8	3	6	6	3	5	4	8
P11	2	0	0	2	2	0	3	1	0	8	6	7
P12	0	2	0	2	0	0	2	0	0	10	5	14
P13	8	0	0	1	8	0	2	3	0	3	4	6
P14	6	4	0	5	6	0	9	3	0	11	5	10
P15	0	0	0	1	2	0	1	0	0	2	1	0
P16	0	0	0	1	1	1	0	0	0	1	1	1
P17	1	0	3	3	1	3	1	0	2	3	0	0
P18	0	1	0	1	0	0	1	1	0	3	1	1
P19	0	0	0	3	0	0	9	6	4	4	3	0
P20	2	3	0	2	2	0	6	5	0	7	6	4
P21	4	3	0	2	4	0	2	3	0	2	3	4
AVG	2.1	0.95	0.57	4.43	2.76	1.19	4.33	2.1	0.86	4.9	2.81	3.57
SD	2.45	1.2	1.12	6.88	2.81	1.69	3.61	2.02	1.35	3.11	1.86	3.78

Regarding the dynamic evaluation, the test suites were augmented using LLMs and manual intervention, which are susceptible to inaccuracies. The subjective nature of human assessment and the inherent limitations of LLMs in understanding and generating contextually accurate test cases may affect the reliability of the test suites. In addition, the authors also manually labeled equivalent mutants, which is also subject to human subjectivity and mistakes.

Moreover, the 21 code structures selected for analysis represent only a fraction of the myriad configurations prevalent in real-world applications. This sample may not comprehensively capture the diversity and complexity of typical Python applications, limiting the generalization of our findings across all possible use scenarios. Such factors should be carefully considered when interpreting the results of our study, as they may influence the determination of the most effective mutation testing tool for Python. However, our intention was to reduce the number of tools used in other Python experiments. In this sense, tools with lower performance on small programs will not perform better on larger programs.

8 Conclusion

Our findings highlight the strengths and limitations of each tool, with Cosmic-Ray demonstrating considerable potential due to its advanced features and customization capabilities. They also show MutPy's capability of generating relevant mutants, which results in effective tests across all analyzed tools. Despite the competitive performance, our cross-examination revealed that even the best test suites have room for improvement, particularly in detecting mutants generated by different tools. The results underscore the importance of continuous updates and community engagement in developing mutation testing tools, as evidenced by the characteristics of tools like Cosmic-Ray and MutMut, which benefit from active contributions. Furthermore, integrating LLMs and manual testing strategies has proven effective in enhancing the comprehensiveness of test suites. However, this approach also introduces challenges related to the accuracy and completeness of the tests.

In conclusion, while each tool has its merits, the choice of a mutation testing tool can often depend on project requirements and the desired level of test customization. Future research should focus on refining the integration of automated and manual test generation techniques and exploring the scalability of these tools in larger, more complex Python projects. Additionally, fostering a more robust community around these tools could enhance their effectiveness and ensure their relevance in the face of evolving software development practices.

Other mutation testing tools for Python may appear in the future, so we can expand the set of mutation testing tools for Python under analysis, improving the test suites for a broader range of projects and making them public available for future investigations. This expansion will enhance the comprehensiveness of our evaluations and provide deeper insights into the adaptability and efficacy of different testing tools across diverse application domains.

Furthermore, augmenting the questionnaire stages with developers is crucial. Enhancing the quality and quantity of developer engagement will yield more detailed and accurate information, facilitating a better understanding of the tools' practical capabilities and limitations. Such efforts are essential for continuously improving mutation testing tools and fostering a more robust ecosystem around software testing methodologies.

It is also relevant to consider the possibility of developing a specific analysis of each tool's mutation operators to identify which are more influential in generating equivalent mutants. This approach could maximize the development of tools capable of generating only mutants relevant to the tested programs with a minimal number of equivalent mutants.

Acknowledgments

This work is partially supported by Brazilian Funding Agencies FAPESP (Grant nº 2019/23160-0 and 2023/00001-9), CAPES (Grant nº 001 and 88887.888653/2023-00), and CNPq (Grant nº 41137/2021-5).

References

[1] Domenico Amalfitano, Ana C. R. Paiva, Alexis Inquel, Luís Pinto, Anna Rita Fasolino, and René Just. 2022. How Do Java Mutation Tools Differ? *Commun. ACM* 65, 12 (Nov. 2022), 74–89. https://doi.org/10.1145/3526099 Place: New York, NY, USA Publisher: Association for Computing Machinery.

[2] J. H. Andrews, L. C. Briand, and Y. Labiche. 2005. Is mutation an appropriate tool for testing experiments?. In *XXVII International Conference on Software Engineering – ICSE'05.* ACM Press, St. Louis, MO, USA, 402–411. https://doi.org/10.1145/1062455.1062530

[3] Austin Bingham. 2021. Cosmic Ray: mutation testing for Python. (Nov. 2021). https://github.com/sixty-north/cosmic-ray

[4] Stéphane Ducasse, Manuel Oriol, and Alexandre Bergel. 2011. Challenges to support automated random testing for dynamically typed languages. In *Proceedings of the International Workshop on Smalltalk Technologies (IWST '11).* Association for Computing Machinery, New York, NY, USA, 1–6. https://doi.org/10.1145/2166929.2166938

[5] Lucca Renato Guerino and Auri M. R. Vincenzi. 2023. An Experimental Study Evaluating Cost, Adequacy Effectiveness of Pynguin's Test Sets. In *8th Brazilian Symposium on Systematic and Automated Software Testing – SAST'2023.* ACM Press, Campo Grande, MS, 5–14. https://doi.org/10.1145/3624032.3624034

[6] Philipp Hossner, Konrad Hałas, Steven Myint, and Andreas Mueller. 2021. MutPy: a mutation testing tool for Python 3.x source code. (Nov. 2021). https://github.com/mutpy/mutpy

[7] Anders Hovmöller. 2021. Mutmut: a Python mutation testing system. (Nov. 2021). https://hackernoon.com/mutmut-a-python-mutation-testing-system-9b9639356c78

[8] Evan Kepner. 2011. Mutatest: Python mutation testing. (Nov. 2011). https://github.com/EvanKepner/mutatest

[9] Marinos Kintis, Mike Papadakis, Andreas Papadopoulos, Evangelos Valvis, Nicos Malevris, and Yves Le Traon. 2018. How effective are mutation testing tools? An empirical analysis of Java mutation testing tools with manual analysis and real faults. *Empirical Software Engineering* 23, 4 (Aug. 2018), 2426–2463. https://doi.org/10.1007/s10664-017-9582-5

[10] Tsz-On Li, Wenxi Zong, Yibo Wang, Haoye Tian, Ying Wang, Shing-Chi Cheung, and Jeff Kramer. 2023. Finding Failure-Inducing Test Cases with ChatGPT.

[11] Stephan Lukasczyk. 2019. Generating Tests to Analyse Dynamically-Typed Programs. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE).* IEEE, San Diego, CA, USA, 1226–1229. https://doi.org/10.1109/ASE.2019.00146 ISSN: 2643-1572.

[12] Stephan Lukasczyk, Florian Kroiß, and Gordon Fraser. 2023. An empirical study of automated unit test generation for Python. *Empirical Software Engineering* 28, 2 (Jan. 2023), 36. https://doi.org/10.1007/s10664-022-10248-w

[13] P. McMinn. 2011. Search-Based Software Testing: Past, Present and Future. In *2011 IEEE Fourth International Conference on Software Testing, Verification and Validation Workshops.* IEEE, Berlin, Germany, 153–163. https://doi.org/10.1109/ICSTW.2011.100 bibtex*[number=] ISSN:.

[14] András Márki and Birgitta Lindström. 2017. Mutation tools for Java. In *Proceedings of the Symposium on Applied Computing (SAC'17).* Association for Computing Machinery, New York, NY, USA, 1364–1415. https://doi.org/10.1145/3019612.3019825

[15] Goran Petrović, Marko Ivanković, Gordon Fraser, and René Just. 2021. Does Mutation Testing Improve Testing Practices?. In *Proceedings of the 43rd International Conference on Software Engineering (ICSE '21).* IEEE Press, Madrid, Spain, 910–921. https://doi.org/10.1109/ICSE43902.2021.00087 Place: Madrid, Spain.

[16] Mohammed Latif Siddiq, Joanna C. S. Santos, Ridwanul Hasan Tanvir, Noshin Ulfat, Fahmid Al Rifat, and Vinicius Carvalho Lopes. 2023. Exploring the Effectiveness of Large Language Models in Generating Unit Tests. https://doi.org/10.48550/arXiv.2305.00418

[17] Zhao Tian, Honglin Shu, Dong Wang, Xuejie Cao, Yasutaka Kamei, and Junjie Chen. 2024. Large Language Models for Equivalent Mutant Detection: How Far Are We?. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2024).* Association for Computing Machinery, New York, NY, USA, 1733–1745. https://doi.org/10.1145/3650212.3680395 event-place: Vienna, Austria.

[18] TIOBE Software BV. 2024. TIOBE Index. https://www.tiobe.com/tiobe-index/

[19] Serhat Uzunbayir and Kaan Kurtel. 2019. An Analysis on Mutation Testing Tools For C# Programming Language. In *2019 4th International Conference on Computer Science and Engineering (UBMK).* IEEE, Samsun, Turkey, 439–443. https://doi.org/10.1109/UBMK.2019.8907222

[20] Sebastian Vogl, Sebastian Schweikl, Gordon Fraser, Andrea Arcuri, Jose Campos, and Annibale Panichella. 2021. EvoSuite at the SBST 2021 Tool Competition. In *2021 IEEE/ACM 14th International Workshop on Search-Based Software Testing (SBST).* IEEE, Madrid, Spain, 28–29. https://doi.org/10.1109/SBST52555.2021.00012

[21] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software Testing with Large Language Models: Survey, Landscape, and Vision. https://arxiv.org/abs/2307.07221 To appear in IEEE Transactions on Software Engineering.

[22] Zhuokui Xie, Yinghao Chen, Chen Zhi, Shuiguang Deng, and Jianwei Yin. 2023. ChatUniTest: a ChatGPT-based automated unit test generation tool. https://doi.org/10.48550/arXiv.2305.04764

[23] Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. 2023. No More Manual Tests? Evaluating and Improving ChatGPT for Unit Test Generation. https://doi.org/10.48550/arXiv.2305.04207